COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	8/27/2026	**DATE SUBMITTED:**	8/27/2026
NAME:	Nichollo Dave A. Galvez	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

ILOA
Main.cpp

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "nodes.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main()
{
    // ========================
    // LINEAR SEARCH - ARRAY
    // ========================

    int dataset[max_size];

    srand(time(0));

    for (int i = 0; i < max_size; i++)
    {
        dataset[i] = rand();
    }

    for (int i = 0; i < max_size; i++)
    {
        std::cout << dataset[i] << " ";
    }

    std::cout << std::endl;

    int dataFind;

    std::cout << "Enter number to search: ";
    std::cin >> dataFind;

    linearSearch(dataset, max_size, dataFind);


    // ========================
    // LINEAR SEARCH - LINKED LIST
    // ========================

```

```
    // Linked list: J I M L O R D
    Node<char>* name1 = new_node('J');
    Node<char>* name2 = new_node('I');
    Node<char>* name3 = new_node('M');
    Node<char>* name4 = new_node('L');
    Node<char>* name5 = new_node('O');
    Node<char>* name6 = new_node('R');
    Node<char>* name7 = new_node('D');

    // Link the nodes together
    name1->next = name2;
    name2->next = name3;
    name3->next = name4;
    name4->next = name5;
    name5->next = name6;
    name6->next = name7;
    name7->next = nullptr;

    std::cout << "Linear Search for Linked List result: ";
    linearSearchLL(name1, 'D');

    std::cout << "Linear Search for Linked List result: ";
    linearSearchLL(name1, 'X');


    // ========================
    // BINARY SEARCH - ARRAY
    // ========================

    int data[] = {10, 20, 30, 40, 50};

    int findData;

    std::cout << "\nBinary Search for Array" << std::endl;
    std::cout << "Enter number to search: ";
    std::cin >> findData;

    binarySearch(data, 5, findData);


```

```
    // ========================
    // BINARY SEARCH - LINKED LIST
    // ========================

    // Ordered linked list: 10 20 30 40 50
    Node<int>* num1 = new_node(10);
    Node<int>* num2 = new_node(20);
    Node<int>* num3 = new_node(30);
    Node<int>* num4 = new_node(40);
    Node<int>* num5 = new_node(50);

    // Link the nodes together
    num1->next = num2;
    num2->next = num3;
    num3->next = num4;
    num4->next = num5;
    num5->next = nullptr;

    // Display linked list
    std::cout << "\nBinary Search for Linked List: ";

    Node<int>* current = num1;

    while (current != nullptr)
    {
        std::cout << current->data << " ";
        current = current->next;
    }

    std::cout << std::endl;

    int findDataLL;

    std::cout << "Enter number to search: ";
    std::cin >> findDataLL;

    binarySearchLL(num1, findDataLL);

    return 0;
}
```

nodes.h

```h
#ifndef NODES_H
#define NODES_H

#include <cstddef> // for NULL / nullptr

// Definition of single linked list node
template <typename T>
class Node {
public:
    T data;
    Node *next;
};

// Function that creates a new node and initializes next pointer to null
template <typename T>
Node<T>* new_node(T newData) {
    Node<T> *newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = NULL;
    return newNode;
}

#endif
```

searchingAlgo.h

```h
#include "nodes.h"
#include <iostream>


#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

template <typename T>
void linearSearch(T data[], int arrSize, T dataFind)
{
    // Step 1: i = 0
    int i = 0;

    // Step 2: Repeat while i < n
    while (i < arrSize)
    {
        // Step 3: If item == data[i]
        if (dataFind == data[i])
        {
            std::cout << "Searching is successful" << std::endl;
            return;
        }

```

```
        // Step 4: i = i + 1
        i++;
    }

    // Step 5: If i == n
    std::cout << "The element is not found" << std::endl;
}

// NODES -> starting node
//  item  that we are searching

template <typename T>
void linearSearchLL(Node<T> *node, T dataFind)
{
    // repeat the loop until the current node is null
    while (node != nullptr)
    {

        // if item == current node's data then display searching is successful
        if (dataFind == node->data)
        {
            std::cout << "the search is success\n";
            return;
        }

        // if not, move to the next node
        node = node->next;
    }

    //the searching reached nullptr, -> search is unsuccessful
    std::cout << "the element is not found \n";
}

template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last) {

```

```
    Node<T>* slow = start;
    Node<T>* fast = start;

    // Fast pointer moves twice as fast as slow pointer
    while (fast != last && fast->next != last) {
        fast = fast->next->next;
        slow = slow->next;
    }

    // Return the slow pointer address as the middle node
    return slow;
}


template <typename T>
void binarySearch(T data[], int arrSize, T dataFind)
{
    int low = 0;
    int up = arrSize - 1;

    while (low <= up)
    {
        int mid = (low + up) / 2;

        if (dataFind == data[mid])
        {
            std::cout << "Search element is found!" << std::endl;
            return;
        }
        else
        {
            if (dataFind < data[mid])
            {
                up = mid - 1;
            }
            else
            {
                low = mid + 1;
```

```
            }
        }
    }

    std::cout << "Search element is not found" << std::endl;
}

template <typename T>
void binarySearchLL(Node<T>* start, T findData) {
    //Start node is set to head of the list and last node is set to NULL
    Node<T>* last = nullptr;

    do {
        // Middle element is calculated using the two pointers approach
        Node<T>* mid = getMiddle(start, last);

        // If sub-list is empty or pointers cross
        if (mid == nullptr) {
            std::cout << "the element is not found \n";
            return;
        }

        // If middle element is same as key to be searched
        if (mid->data == findData) {
            std::cout << "the search is success\n";
            return;
        }
        // If middle element is greater than key, search right side (or if key > mid->data depending on sort order)
        // Following logic for standard ascending sorted list:
        else if (mid->data < findData) {
            start = mid->next; // Set start pointer to next of middle element
        }
        // If middle element is less than key, search left side
        else {
            last = mid; // Set last pointer to middle element
        }
```

```
        // 6. Loop breaks when start reaches or passes last node
        } while (last == nullptr || last != start);

        std::cout << "the element is not found \n";
    }

    #endif
```

```
PS D:\For Academic Purposes\CPE21S1_GALVEZ\HOA6.1\procedure> cd "d:\For Academic Purposes\CPE21S1_GALVEZ\HOA6.1\procedure\" ; if ($?) { g++ main.cpp -o main
14737 18308 21302 23598 5900 5966 11622 27191 11113 25105 27603 12301 16079 16482 12686 15885 29563 4515 20037 7105 3866 12244 29378 14934 29558 5548 3764 21
156 10398 10980 22909 29222 11454 22888 16261 1165 23626 31172 10159 28052 17638 15391 13910 25018 9317 5388
Enter number to search: 5
The element is not found
Linear Search for Linked List result: the search is success
Linear Search for Linked List result: the element is not found

Binary Search for Array
Enter number to search: 10
Search element is found!

Binary Search for Linked List: 10 20 30 40 50
Enter number to search: 144
the element is not found
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILOB

Problem 1.

```cpp
#include <iostream>

int main()
{
    int data[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int key = 18;
    int comparisons = 0;

    for (int i = 0; i < 10; i++)
    {
        comparisons++;

        if (data[i] == key)
        {
            std::cout << "Found: " << data[i] << std::endl;
            break;
        }
    }

    std::cout << "Comparisons: " << comparisons << std::endl;

    return 0;
}
```

```
Found: 18
Comparisons: 2
PS D:\For Academic Purposes\CPE21S1_GALVEZ\HOA6.1\ILOB>
```

Problem 2

```cpp
#include <iostream>

int countRepeating(int data[], int size, int key)
{
    int count = 0;

    for (int i = 0; i < size; i++)
    {
        if (data[i] == key)
        {
            count++;
        }
    }

    return count;
}

int main()
{
    int data[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    int key = 18;

    int result = countRepeating(data, 10, key);

    std::cout << "Number of occurrences of "
              << key << ": " << result << std::endl;

    return 0;
}
```

```
PS D:\For Academic Purposes\CPE21S1_GALVEZ\HOA6.1\ILOB> cd "
Found: 18
Comparisons: 2
PS D:\For Academic Purposes\CPE21S1_GALVEZ\HOA6.1\ILOB>
```

Problem 3

```cpp
#include <iostream>

int main()
{
    int data[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};

    int key = 8;
    int low = 0;
    int up = 9;
    int comparisons = 0;

    while (low <= up)
    {
        int mid = (low + up) / 2;

        comparisons++;

        std::cout << "Iteration " << comparisons
                  << ": mid = " << data[mid] << std::endl;

        if (data[mid] == key)
        {
            std::cout << "Search element is found!" << std::endl;
            std::cout << "Index: " << mid << std::endl;
            std::cout << "Comparisons: " << comparisons << std::endl;
            return 0;
        }
        else if (key < data[mid])
        {
            up = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }
    std::cout << "Search element is not found" << std::endl;
    return 0;
}
```

```
Iteration 1: mid = 11
Iteration 2: mid = 5
Iteration 3: mid = 6
Iteration 4: mid = 8
Search element is found!
Index: 3
Comparisons: 4
PS D:\For Academic Purposes\CPE21S1_GALVEZ\HOA6.1\ILOB>
```

Problem 4

```cpp
#include <iostream>

int recursiveBinarySearch(int data[], int low, int up, int key)
{
    if (low > up)
    {
        return -1;
    }

    int mid = (low + up) / 2;

    std::cout << "Checking: " << data[mid] << std::endl;

    if (data[mid] == key)
    {
        return mid;
    }

    if (key < data[mid])
    {
        return recursiveBinarySearch(data, low, mid - 1, key);
    }

    return recursiveBinarySearch(data, mid + 1, up, key);
}

int main()
{
    int data[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};

    int key = 8;

    int result = recursiveBinarySearch(data, 0, 9, key);

    if (result != -1)
    {
        std::cout << "Search element is found!" << std::endl;
        std::cout << "Index: " << result << std::endl;
    }
    else
```

```
41      {
42          std::cout << "Search element is not found" << std::endl;
43      }
44
45      return 0;
46  }
```

```
Checking: 11
Checking: 5
Checking: 6
Checking: 8
Search element is found!
Index: 3
```

Conclusion

Summary of Lessons Learned

In this activity, I gained hands-on experience implementing Linear Search and Binary Search on both arrays and linked lists in C++. I saw firsthand how Linear Search processes data sequentially regardless of order, while Binary Search relies on sorted elements to rapidly narrow down the search space by half.

Analysis of the Procedure

Working through the procedural steps showed me the key trade-offs between array-based and linked list-based search implementations. While arrays allow direct index access for fast middle-element calculations in Binary Search, linked lists require using a two-pointer approach (slow and fast pointers) to locate the midpoint.

Analysis of the Supplementary Activity

Solving the supplementary problems deepened my understanding of algorithm performance and adaptability. Manually counting comparisons for Linear Search reinforced how inefficient sequential traversal gets as datasets grow, while modifying Binary Search into a recursive function helped me appreciate how structural changes can simplify search logic.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation, summarization**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.